

LAPORAN PRATIKUM STRUKTUR DATA
SORTING DENGAN BAHASA JAVA



Dosen Pengampu:
Wahyudi Dr.. S.T. M.T.

Disusun Oleh:
Kevin Maulana Sempri
2511531019

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG
2026

Kata Pengantar

Puji syukur penulis ucapkan kepada Allah SWT karena penulis dapat menyelesaikan laporan praktikum ini dengan judul "Sorting Dengan Bahasa Java" dengan tepat waktu. Laporan ini dibuat sebagai salah satu untuk memenuhi nilai praktikum pada mata Struktur Data. Laporan ini berfokus pada studi kasus Sorting dengan menggunakan Bahasa Java. Penulis menyadari bahwa laporan praktikum ini masih jauh dari sempurna, oleh karena itu penulis mengucapkan banyak terima kasih kepada Bapak / Ibu dosen dan para asisten, serta rekan-rekan mahasiswa yang telah banyak membantu penulis, dalam memberikan arahan & ilmu. Untuk perbaikan & kemajuan di masa yang akan datang, penulis mengharapkan kritik & saran yang membangun. Semoga laporan ini bisa memberikan manfaat & wawasan bagi pembaca, khususnya pada studi kasus Sorting menggunakan Bahasa Java. Akhir kata, penulis ucapkan terima kasih kepada semuanya.

DAFTAR PUSTAKA

KATA PENGANTAR	i
DAFTAR ISI	ii
DAFTAR PUSTAKA	iii
BAB I PENDAHULUAN	1
1.1 Latar Belakang.....	1
1.2 Tujuan Pratikum.....	1
1.3 Manfaat Pratikum.....	1
BAB II HASIL DAN PEMBAHASAN.....	2
2.1 Waktu dan Tempat.....	2
2.2 Cara Kerja.....	2
2.3 Hasil.....	3
2.4 Pembahasan	7
DAFTAR PUSTAKA.....	

Bab 1 Pendahuluan

1.1 Latar Belakang

Perkembangan teknologi informasi menuntut adanya sistem penyimpanan data yang terstruktur dan mudah diakses. Struktur Data menjadi salah satu komponen penting dalam pengelolaan informasi, baik di bidang pendidikan, bisnis, maupun pemerintahan. Eclipse IDE merupakan salah satu perangkat lunak yang menyediakan fasilitas untuk merancang, mengelola, dan memanipulasi kode Bahasa pemrograman Java yang relatif mudah dipahami. Melalui praktikum ini, diharapkan mampu memahami Struktur Data Linked List serta mengimplementasikan studi kasus dengan menggunakan Bahasa Java.

1.2 Tujuan praktikum

1. Memahami konsep dasar struktur data dengan tipe data abstrak Sorting.
2. Mempelajari cara membuat struktur data Sorting dengan menggunakan Bahasa Java.
3. Mengimplementasikan struktur data sesuai kebutuhan studi kasus.
4. Melatih keterampilan dalam mengelola data secara sistematis dan efisien.

1.3 Manfaat praktikum

1. Memberikan pengalaman langsung dalam merancang dan mengelola struktur data.
2. Menambah wawasan tentang penggunaan struktur data dengan menggunakan Bahasa Java.
3. Mempersiapkan mahasiswa dengan keterampilan praktis yang dapat diterapkan dalam dunia kerja.
4. Membantu memahami pentingnya integritas dan konsistensi data dalam sistem informasi.

Bab 2 Hasil dan Pembahasan

Pada bab ini saya akan menampilkan hasil praktikum yang telah dilaksanakan beserta pembahasannya

2.1 Waktu dan Tempat

Praktikum ini dilaksanakan pada hari Senin tanggal 2 Juni 2026 Dilaksanakan di laboratorium Jurusan Informatika, Fakultas Teknologi Infomarsi, Universitas Andalas.

2.2 Cara Kerja

Praktikan akan membuat sebuah program yang berupa Bahasa Pemrograman Java. Perintahnya adalah membuat kode untuk Abstract Data Types (Tipe data abstrak) dan kode untuk Driver (memanggil ADT tersebut)

2.3 Hasil

Kode Java ShellSort:

```
package pekan8_2511531019;
public class ShellSort_2511531019 {
    public static void shellSort_2511531019(int[] A_1019) {
        int n_1019 = A_1019.length;
        int gap_1019 = n_1019 / 2;
        while (gap_1019 > 0) {
            for (int i_1019 = gap_1019; i_1019 < n_1019;
i_1019++) {
                int temp_1019 = A_1019[i_1019];
                int j_1019 = i_1019;
                while (j_1019 >= gap_1019 && A_1019[j_1019
- gap_1019] > temp_1019) {
                    A_1019[j_1019] = A_1019[j_1019 -
gap_1019];
                    j_1019 = j_1019 - gap_1019;
                }
                A_1019[j_1019] = temp_1019;
            }
            gap_1019 = gap_1019 / 2;
        }
    }

    public static void printArray_2511531019(int[] arr_1019) {
        for (int i_1019 : arr_1019) {
            System.out.print(i_1019 + " ");
        }
        System.out.println();
    }
}
```

```

    public static void main(String[] args) {
        // TODO Auto-generated method stub
        int[] data_1019 = {3, 10, 4, 6, 8, 9, 7, 2, 1, 5};
        System.out.print("Sebelum: ");
        printArray_2511531019(data_1019);

        shellSort_2511531019(data_1019);

        System.out.print("Sesudah (Shell Sort): ");
        printArray_2511531019(data_1019);
    }
}

```

Kode Java QuickSort:

```

package pekan8_2511531019;

public class QuickSort_2511531019 {
    static void swap_2511531019(int[] arr_1019, int i_1019, int j_1019) {
        int temp_1019 = arr_1019[i_1019];
        arr_1019[i_1019] = arr_1019[j_1019];
        arr_1019[j_1019] = temp_1019;
    }

    static void medianofthree_2511531019(int[] arr_1019, int low_1019, int high_1019) {
        int mid_1019 = low_1019 + (high_1019 - low_1019) / 2;
        if (arr_1019[low_1019] > arr_1019[mid_1019]) {
            swap_2511531019(arr_1019, low_1019, mid_1019);
        }
        if (arr_1019[low_1019] > arr_1019[high_1019]) {
            swap_2511531019(arr_1019, low_1019, high_1019);
        }
        if (arr_1019[mid_1019] > arr_1019[high_1019]) {
            swap_2511531019(arr_1019, mid_1019, high_1019);
        }
        swap_2511531019(arr_1019, mid_1019, high_1019);
    }

    static int partition_2511531019(int[] arr_1019, int low_1019, int high_1019) {
        int pivot_1019 = arr_1019[high_1019];
        int i_1019 = (low_1019 - 1);
        for (int j_1019 = low_1019; j_1019 <= high_1019 - 1; j_1019++) {
            if (arr_1019[j_1019] < pivot_1019) {
                i_1019++;
                swap_2511531019(arr_1019, i_1019, j_1019);
            }
        }
        swap_2511531019(arr_1019, i_1019 + 1, high_1019);
        return (i_1019 + 1);
    }
}

```

```

        static void quickSort_2511531019(int[] arr_1019, int low_1019, int
high_1019) {
            if (low_1019 < high_1019) {
                int pi_1019 = partition_2511531019(arr_1019, low_1019,
high_1019);

                quickSort_2511531019(arr_1019, low_1019, pi_1019 - 1);
                quickSort_2511531019(arr_1019, pi_1019 + 1, high_1019);
            }
        }

        public static void printArr_2511531019(int[] arr) {
            for (int i_1019 = 0; i_1019 < arr.length; i_1019++) {
                System.out.print(arr[i_1019] + " ");
            }
            System.out.println();
        }

        public static void main(String[] args) {
            int[] arr_1019 = {10, 7, 8, 9, 1, 5};
            int N_1019 = arr_1019.length;
            System.out.print("Data sebelum diurutkan: ");
            printArr_2511531019(arr_1019);

            quickSort_2511531019(arr_1019, 0, N_1019 - 1);
            System.out.print("Data terurut quick sort: ");
            printArr_2511531019(arr_1019);
        }
    }
}

```

Kode Java MergeSort:

```

package pekan8_2511531019;

public class MergeSort_2511531019 {
    void merge_2511531019(int arr_1019[], int l_1019, int m_1019, int
r_1019) {
        int n1_1019 = m_1019 - l_1019 + 1;
        int n2_1019 = r_1019 - m_1019;

        int L_1019[] = new int[n1_1019];
        int R_1019[] = new int[n2_1019];

        for (int i_1019 = 0; i_1019 < n1_1019; ++i_1019) {
            L_1019[i_1019] = arr_1019[l_1019 + i_1019];
        }
        for (int j_1019 = 0; j_1019 < n2_1019; ++j_1019) {
            R_1019[j_1019] = arr_1019[m_1019 + 1 + j_1019];
        }
        int i_1019 = 0, j_1019 = 0;

        int k_1019 = l_1019;
        while (i_1019 < n1_1019 && j_1019 < n2_1019) {
            if (L_1019[i_1019] <= R_1019[j_1019]) {
                arr_1019[k_1019] = L_1019[i_1019];
                i_1019++;
            } else {
                arr_1019[k_1019] = R_1019[j_1019];
            }
        }
    }
}

```

```

        j_1019++;
    }
    k_1019++;
}

while (i_1019 < n1_1019) {
    arr_1019[k_1019] = L_1019[i_1019];
    i_1019++;
    k_1019++;
}

while (j_1019 < n2_1019) {
    arr_1019[k_1019] = R_1019[j_1019];
    j_1019++;
    k_1019++;
}
}

void sort_2511531019(int arr_1019[], int l_1019, int r_1019) {
    if (l_1019 < r_1019) {
        int m_1019 = (l_1019 + r_1019) / 2;

        sort_2511531019(arr_1019, l_1019, m_1019);
        sort_2511531019(arr_1019, m_1019 + 1, r_1019);

        merge_2511531019(arr_1019, l_1019, m_1019, r_1019);
    }
}

static void printArray_2511531019(int arr_1019[]) {
    int n_1019 = arr_1019.length;
    for (int i_1019 = 0; i_1019 < n_1019; ++i_1019) {
        System.out.print(arr_1019[i_1019] + " ");
    }
    System.out.println();
}

public static void main(String[] args) {
    int arr_1019[] = {12, 11, 13, 5, 6, 7};
    System.out.println("Sebelum terurut: ");
    printArray_2511531019(arr_1019);

    MergeSort_2511531019 ob_1019 = new MergeSort_2511531019();
    ob_1019.sort_2511531019(arr_1019, 0, arr_1019.length - 1);
    System.out.println("\nSesudah terurut menggunakan merge sort:");
    printArray_2511531019(arr_1019);
}
}

```

Kode Java BubbleSortGUI:

```

package pekan8_2511531019;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;

```



```

import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;
import java.lang.reflect.Array;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class BubbleSortGUI_2511531019 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_1019;
    private JLabel[] labelArray_1019;
    private JButton stepButton_1019, resetButton_1019, setButton_1019;
    private JTextField inputField_1019;
    private JPanel panelArray_1019;
    private JTextArea stepArea_1019;

    private int i_1019 = 1, j_1019;
    private boolean sorting_1019 = false;
    private int stepCount_1019 = 1;

    /**
     * Launch the application.
     */
    /**
     public static void main(String[] args) {
         EventQueue.invokeLater(new Runnable() {
             public void run() {
                 try {
                     BubbleSortGUI_2511531019 frame = new
BubbleSortGUI_2511531019();
                     frame.setVisible(true);
                 } catch (Exception e) {
                     e.printStackTrace();
                 }
             }
         });
     }
    */

    /**
     * Create the frame.
     */
    public BubbleSortGUI_2511531019() {
        setTitle("Bubble Sort Langkah per Langkah");
        setSize(750, 400);
    }
}

```

```

        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel Layout
        JPanel inputPanel = new JPanel(new FlowLayout());
        inputField_1019 = new JTextField(30);
        setButton_1019 = new JButton("Set Array");
        inputPanel.add(inputField_1019);
        inputPanel.add(setButton_1019);

        // Panel Array visual
        panelArray_1019 = new JPanel();
        panelArray_1019.setLayout(new FlowLayout());

        // Panel Kontrol
        JPanel controlPanel = new JPanel();
        stepButton_1019 = new JButton("Langkah Selanjutnya");
        resetButton_1019 = new JButton("Reset");
        stepButton_1019.setEnabled(false);
        controlPanel.add(stepButton_1019);
        controlPanel.add(resetButton_1019);

        // Area Teks untuk log langkah langkah
        stepArea_1019 = new JTextArea(8, 60);
        stepArea_1019.setEditable(false);
        stepArea_1019.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane = new JScrollPane(stepArea_1019);

        // Tambahkan panel ke frame
        add(inputPanel, BorderLayout.NORTH);
        add(panelArray_1019, BorderLayout.CENTER);
        add(controlPanel, BorderLayout.SOUTH);
        add(scrollPane, BorderLayout.EAST);

        // Event Set Array
        setButton_1019.addActionListener(e -> setArrayfromInput());

        // Event Langkah Selanjutnya
        stepButton_1019.addActionListener(e -> performStep());

        // Event Reset
        resetButton_1019.addActionListener(e -> reset());
    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            BubbleSortGUI_2511531019 gui = new
BubbleSortGUI_2511531019();
            gui.setVisible(true);
        });
    }

    private void reset() {
        // TODO Auto-generated method stub
        inputField_1019.setText("");
        panelArray_1019.removeAll();
    }

```

```

        panelArray_1019.revalidate();
        panelArray_1019.repaint();
        stepArea_1019.setText("");
        stepButton_1019.setEnabled(false);
        sorting_1019 = false;
        i_1019 = 0;
        j_1019 = 0;
        stepCount_1019 = 1;
    }

    private void performStep() {
        // TODO Auto-generated method stub
        if (!sorting_1019 || i_1019 >= array_1019.length - 1) {
            sorting_1019 = false;
            stepButton_1019.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting selesai!");
            return;
        }

        resetHighlights();

        StringBuilder stepLog_1019 = new StringBuilder();

        labelArray_1019[j_1019].setBackground(Color.CYAN);
        labelArray_1019[j_1019 + 1].setBackground(Color.CYAN);

        if (array_1019[j_1019] > array_1019[j_1019 + 1]) {
            // Swap
            int temp_1019 = array_1019[j_1019];
            array_1019[j_1019] = array_1019[j_1019 + 1];
            array_1019[j_1019 + 1] = temp_1019;

            labelArray_1019[j_1019].setBackground(Color.RED);
            labelArray_1019[j_1019 + 1].setBackground(Color.RED);

            stepLog_1019.append("Langkah ").append(stepCount_1019)
                .append(": Menukar elemen ke-")
                .append(j_1019).append(" ")
                .append(array_1019[j_1019 + 1]).append(" dengan ke-")
                .append(j_1019 + 1).append(" ")
                .append(array_1019[i_1019]).append("\n");
        } else {
            stepLog_1019.append("Langkah ").append(stepCount_1019)
                .append(": Tidak ada pertukaran antara ke-")
                .append(j_1019).append(" dan ke-")
                .append(j_1019 + 1).append("\n");
        }

        stepLog_1019.append("Hasil: ")
            .append(arrayToString(array_1019))
            .append("\n\n");

        stepArea_1019.append(stepLog_1019.toString());

        updateLabels();

        j_1019++;
    }

```

```

        if (j_1019 >= array_1019.length - i_1019 - 1) {
            j_1019 = 0;
            i_1019++;
        }

        stepCount_1019++;

        if (i_1019 >= array_1019.length - 1) {
            sorting_1019 = false;
            stepButton_1019.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting selesai!");
        }
    }

    private void resetHighlights() {
        // TODO Auto-generated method stub
        for (JLabel label_1019 : labelArray_1019) {
            label_1019.setBackground(Color.WHITE);
        }
    }

    private String arrayToString(int[] arr_1019) {
        // TODO Auto-generated method stub
        StringBuilder sb_1019 = new StringBuilder();
        for (int k_1019 = 0; k_1019 < arr_1019.length; k_1019++) {
            sb_1019.append(arr_1019[k_1019]);
            if (k_1019 < arr_1019.length - 1) sb_1019.append(", ");
        }
        return sb_1019.toString();
    }

    private void updateLabels() {
        // TODO Auto-generated method stub
        for (int k_1019 = 0; k_1019 < array_1019.length; k_1019++) {
            labelArray_1019[k_1019].setText(String.valueOf(array_1019[k_1019]));
        }
    }

    private void setArrayfromInput() {
        // TODO Auto-generated method stub
        String text_1019 = inputField_1019.getText().trim();
        if (text_1019.isEmpty()) return;
        String[] parts_1019 = text_1019.split(",");
        array_1019 = new int [parts_1019.length];
        try {
            for (int k_1019 = 0; k_1019 < parts_1019.length;
k_1019++) {
                array_1019[k_1019] =
Integer.parseInt(parts_1019[k_1019].trim());
            }
        } catch (NumberFormatException e) {

```

```

JOptionPane.showMessageDialog(this, "Masukkan hanya
angka yang dipisahkan " + "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
        return;
    }
    i_1019 = 0;
    j_1019 = 0;
    stepCount_1019 = 1;
    sorting_1019 = true;
    stepButton_1019.setEnabled(true);
    stepArea_1019.setText("");
    panelArray_1019.removeAll();
    labelArray_1019 = new JLabel[array_1019.length];
    for (int k_1019 = 0; k_1019 < array_1019.length; k_1019++) {
        labelArray_1019[k_1019] = new
JLabel(String.valueOf(array_1019[k_1019]));
        labelArray_1019[k_1019].setFont(new Font("Arial",
Font.BOLD, 24));

        labelArray_1019[k_1019].setOpaque(true);
        labelArray_1019[k_1019].setBackground(Color.WHITE);

        labelArray_1019[k_1019].setBorder(BorderFactory.createLineBorder(Color.BLACK));
        labelArray_1019[k_1019].setPreferredSize(new
Dimension(50, 50));

        labelArray_1019[k_1019].setHorizontalAlignment(SwingConstants.CENTER)
;
        panelArray_1019.add(labelArray_1019[k_1019]);
    }
    panelArray_1019.revalidate();
    panelArray_1019.repaint();
}
}

```

Kode Java MergeSortGUI:

```

package pekan8_2511531019;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;
import java.util.LinkedList;
import java.util.Queue;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;

```

```

import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class MergeSortGUI_2511531019 extends JFrame {

    private static final long serialVersionUID = 1L;
    // private JPanel contentPane;
    private int[] array_1019;
    private JLabel[] labelArray_1019;
    private JButton stepButton_1019, resetButton_1019, setButton_1019;
    private JTextField inputField_1019;
    private JPanel panelArray_1019;
    private JTextArea stepArea_1019;

    private Queue<int[]> mergeQueue_1019 = new LinkedList<>();
    private boolean isMerging_1019 = false;
    private boolean copying_1019 = false;
    private int left_1019, mid_1019, right_1019;
    private int[] temp_1019;
    private int k_1019;
    private int i_1019 = 1, j_1019;
    private int stepCount_1019 = 1;

    /**
     * Launch the application.
     */
    /**
     public static void main(String[] args) {
         EventQueue.invokeLater(new Runnable() {
             public void run() {
                 try {
                     MergeSortGUI_2511531019 frame = new
MergeSortGUI_2511531019();
                     frame.setVisible(true);
                 } catch (Exception e) {
                     e.printStackTrace();
                 }
             }
         });
     }
    */

    /**
     * Create the frame.
     */
    public MergeSortGUI_2511531019() {
        /*
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setBounds(100, 100, 450, 300);
        contentPane = new JPanel();
        contentPane.setBorder(new EmptyBorder(5, 5, 5, 5));
        setContentPane(contentPane);
        */
        setTitle("Merge Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
}

```

```

        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel Layout
        JPanel inputPanel = new JPanel(new FlowLayout());
        inputField_1019 = new JTextField(30);
        setButton_1019 = new JButton("Set Array");
        inputPanel.add(inputField_1019);
        inputPanel.add(setButton_1019);

        // Panel Array visual
        panelArray_1019 = new JPanel();
        panelArray_1019.setLayout(new FlowLayout());

        // Panel Kontrol
        JPanel controlPanel = new JPanel();
        stepButton_1019 = new JButton("Langkah Selanjutnya");
        resetButton_1019 = new JButton("Reset");
        stepButton_1019.setEnabled(false);
        controlPanel.add(stepButton_1019);
        controlPanel.add(resetButton_1019);

        // Area Teks untuk log langkah langkah
        stepArea_1019 = new JTextArea(8, 60);
        stepArea_1019.setEditable(false);
        stepArea_1019.setFont(new Font("Monospaced", Font.PLAIN,
14));

        JScrollPane scrollPane = new JScrollPane(stepArea_1019);

        // Tambahkan panel ke frame
        add(inputPanel, BorderLayout.NORTH);
        add(panelArray_1019, BorderLayout.CENTER);
        add(controlPanel, BorderLayout.SOUTH);
        add(scrollPane, BorderLayout.EAST);

        // Event Set Array
        setButton_1019.addActionListener(e -> setArrayfromInput());

        // Event Langkah Selanjutnya
        stepButton_1019.addActionListener(e -> performStep());

        // Event Reset
        resetButton_1019.addActionListener(e -> reset());

    }

    public static void main(String[] args) {
        SwingUtilities.invokeLater(() -> {
            MergeSortGUI_2511531019 gui = new MergeSortGUI_2511531019();
            gui.setVisible(true);
        });
    }

    private void setArrayfromInput() {
        // TODO Auto-generated method stub
        String text_1019 = inputField_1019.getText().trim();

```

```

        if (text_1019.isEmpty()) return;

        String[] parts_1019 = text_1019.split(" ");
        array_1019 = new int[parts_1019.length];

        try {
            for (int i_1019 = 0; i_1019 < parts_1019.length; i_1019++) {
                array_1019[i_1019] =
Integer.parseInt(parts_1019[i_1019].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(
                this,
                "Masukkan hanya angka!",
                "Error",
                JOptionPane.ERROR_MESSAGE
            );

            return;
        }

        labelArray_1019 = new JLabel[array_1019.length];
        panelArray_1019.removeAll();

        for (int i_1019 = 0; i_1019 < array_1019.length; i_1019++) {
            labelArray_1019[i_1019] = new
JLabel(String.valueOf(array_1019[i_1019]));

            labelArray_1019[i_1019].setFont(
                new Font("Arial", Font.BOLD, 24)
            );

            labelArray_1019[i_1019].setOpaque(true);
            labelArray_1019[i_1019].setBackground(Color.WHITE);

            labelArray_1019[i_1019].setBorder(
                BorderFactory.createLineBorder(Color.BLACK)
            );

            labelArray_1019[i_1019].setPreferredSize(
                new Dimension(50, 50)
            );

            labelArray_1019[i_1019].setHorizontalAlignment(
                SwingConstants.CENTER
            );

            panelArray_1019.add(labelArray_1019[i_1019]);
        }

        mergeQueue_1019.clear();

        generateMergeSteps_1019(0, array_1019.length - 1);

        stepButton_1019.setEnabled(true);
        stepArea_1019.setText("");
        stepCount_1019 = 1;

```



```

        isMerging_1019 = false;

        panelArray_1019.revalidate();
        panelArray_1019.repaint();

    }

    private void performStep() {
        // TODO Auto-generated method stub
        resetHighlights();

        if (!isMerging_1019 && !mergeQueue_1019.isEmpty()) {
            int[] range_1019 = mergeQueue_1019.poll();
            left_1019 = range_1019[0];
            mid_1019 = range_1019[1];
            right_1019 = range_1019[2];

            temp_1019 = new int[right_1019 - left_1019 + 1];

            i_1019 = left_1019;
            j_1019 = mid_1019 + 1;
            k_1019 = 0;

            copying_1019 = false;
            isMerging_1019 = true;

            stepArea_1019.append("Langkah " + stepCount_1019++ +
                ": Mulai merge dari " + left_1019 + " ke " + right_1019
+ "\n");

            return;
        }

        if (isMerging_1019 && !copying_1019) {
            if (i_1019 <= mid_1019 && j_1019 <= right_1019) {
                labelArray_1019[i_1019].setBackground(Color.CYAN);
                labelArray_1019[j_1019].setBackground(Color.CYAN);

                if (array_1019[i_1019] <= array_1019[j_1019]) {
                    temp_1019[k_1019++] = array_1019[i_1019++];
                } else {
                    temp_1019[k_1019++] = array_1019[j_1019++];
                }

                stepArea_1019.append("Langkah " + stepCount_1019++ +
                    ": Bandingkan dan salin elemen\n");

                return;
            } else if (i_1019 <= mid_1019) {
                temp_1019[k_1019++] = array_1019[i_1019++];

                stepArea_1019.append("Langkah " + stepCount_1019++ +
                    ": Salin sisa kiri\n");

                return;
            }
        }
    }

```

```

        } else if (j_1019 <= right_1019) {
            temp_1019[k_1019++] = array_1019[j_1019++];

            stepArea_1019.append("Langkah " + stepCount_1019++ +
                ": Salin sisa kanan\n");

            return;
        } else {
            copying_1019 = true;
            k_1019 = 0;

            return;
        }
    }

    if (copying_1019 && k_1019 < temp_1019.length) {
        array_1019[left_1019 + k_1019] = temp_1019[k_1019];
        labelArray_1019[left_1019 +
k_1019].setText(String.valueOf(temp_1019[k_1019]));
        labelArray_1019[left_1019 +
k_1019].setBackground(Color.GREEN);
        stepArea_1019.append("Langkah " + stepCount_1019++ + ":
Tempelkan ke array utama\n");
        k_1019++;
        return;
    }

    if (copying_1019 && k_1019 == temp_1019.length) {
        isMerging_1019 = false;
        copying_1019 = false;
    }

    if (mergeQueue_1019.isEmpty() && !isMerging_1019) {
        stepArea_1019.append("Selesai.\n");
        stepButton_1019.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
    }
}

private void resetHighlights() {
    // TODO Auto-generated method stub
    if (labelArray_1019 == null) return;
    for (JLabel label_1019 : labelArray_1019) {
        label_1019.setBackground(Color.WHITE);
    }
}

private void generateMergeSteps_1019(int left_1019, int right_1019)
{
    if (left_1019 < right_1019) {
        int mid_1019 = (left_1019 + right_1019) / 2;
        generateMergeSteps_1019(left_1019, mid_1019);
        generateMergeSteps_1019(mid_1019 + 1, right_1019);
        mergeQueue_1019.add(new int[]{left_1019, mid_1019,
right_1019});
    }
}

```

```

    }
}

private void reset() {
    // TODO Auto-generated method stub
    inputField_1019.setText("");
    panelArray_1019.removeAll();
    panelArray_1019.revalidate();
    panelArray_1019.repaint();
    stepArea_1019.setText("");
    stepButton_1019.setEnabled(false);
    mergeQueue_1019.clear();
    isMerging_1019 = false;
    stepCount_1019 = 1;
}
}

```

Output Kode Java ShellSort

```

Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (Shell Sort): 1 2 3 4 5 6 7 8 9 10

```

Output Kode Java QuickSort

```

Data sebelum diurutkan: 10 7 8 9 1 5
Data terurut quick sort: 1 5 7 8 9 10

```

Output Kode Java MergeSort

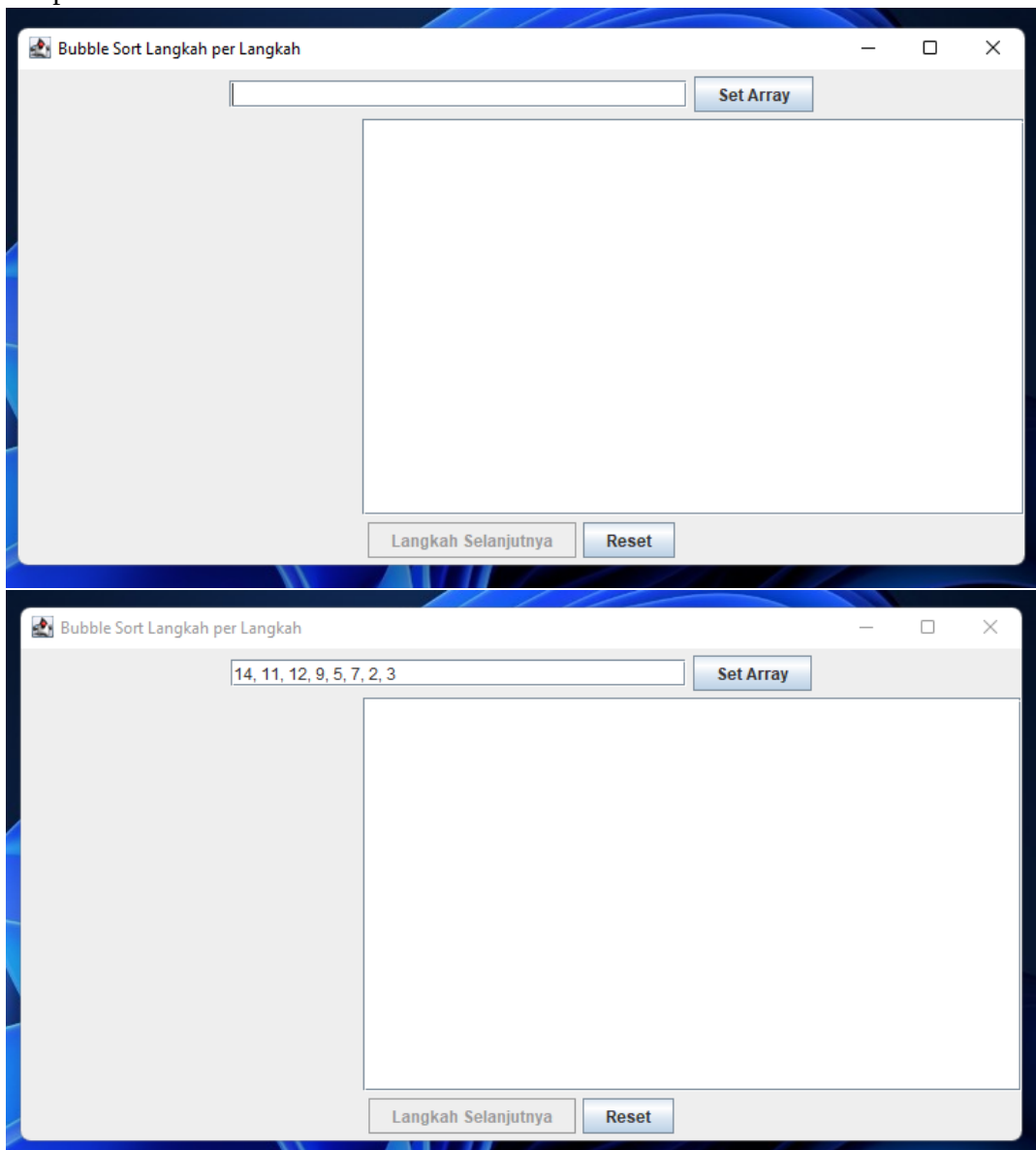
```

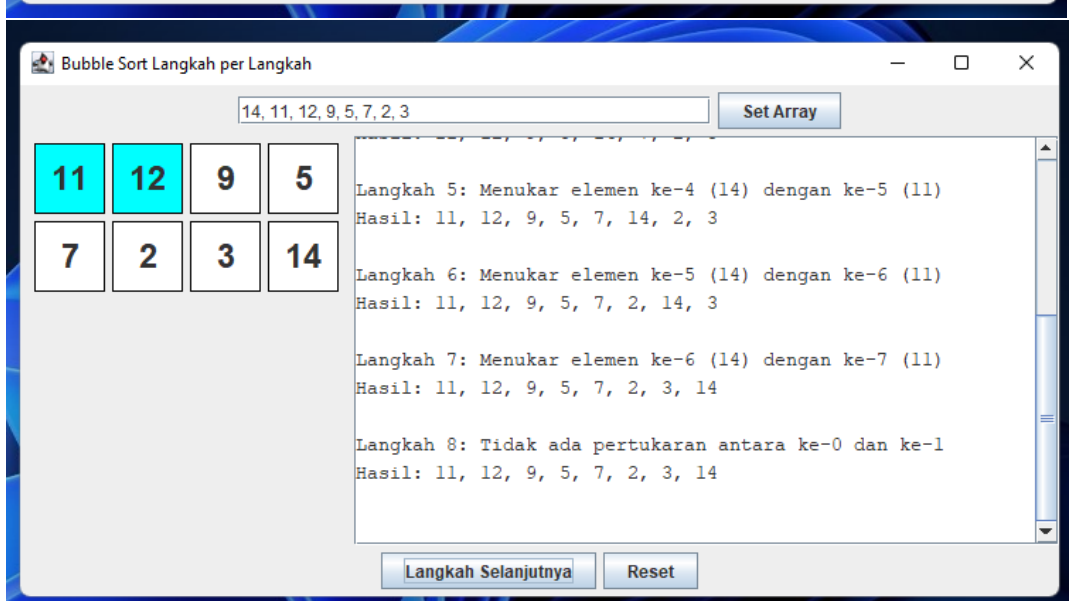
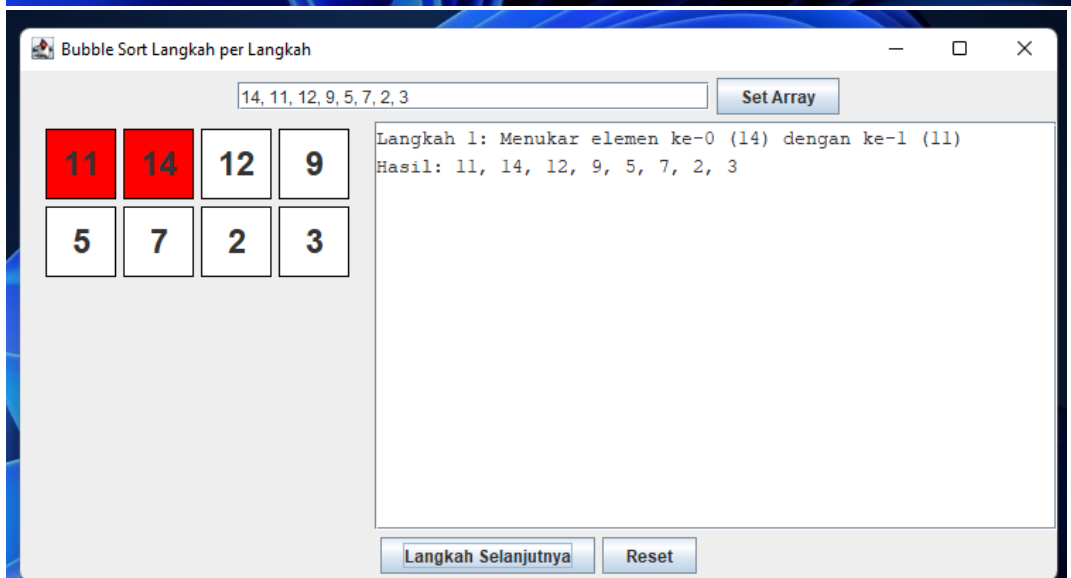
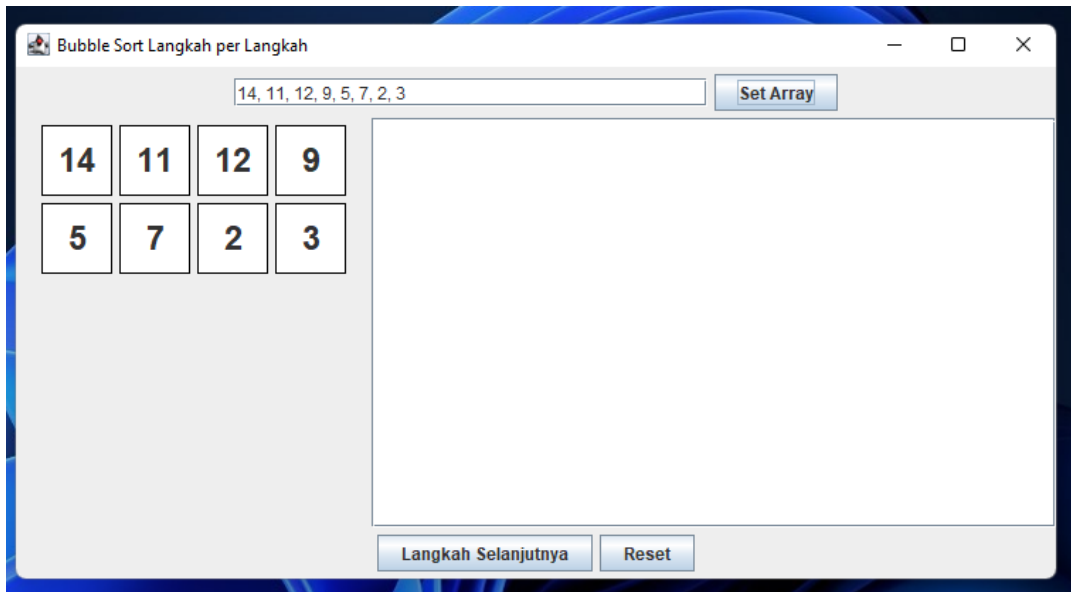
Sebelum terurut:
12 11 13 5 6 7

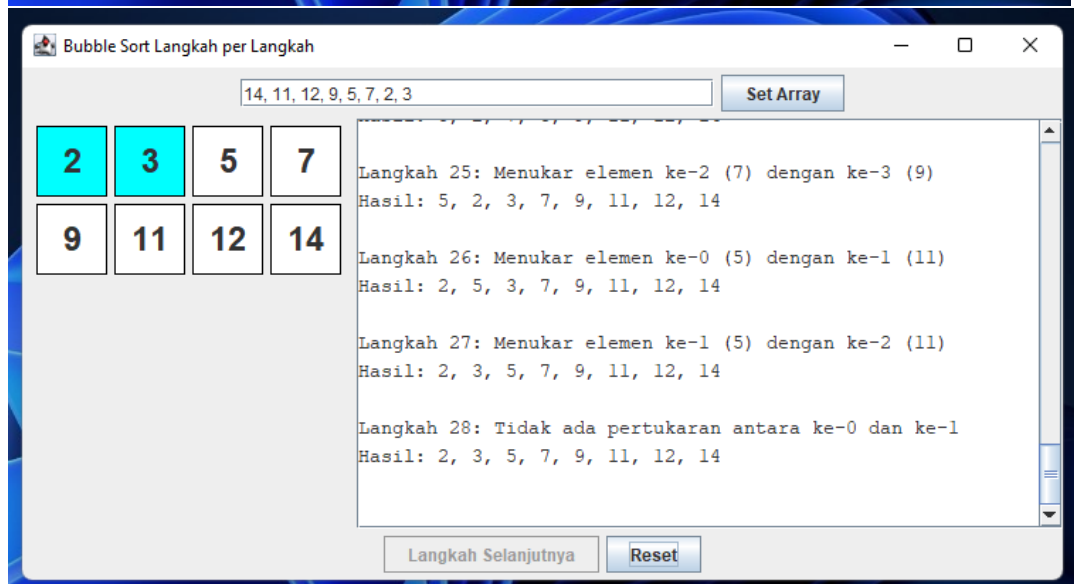
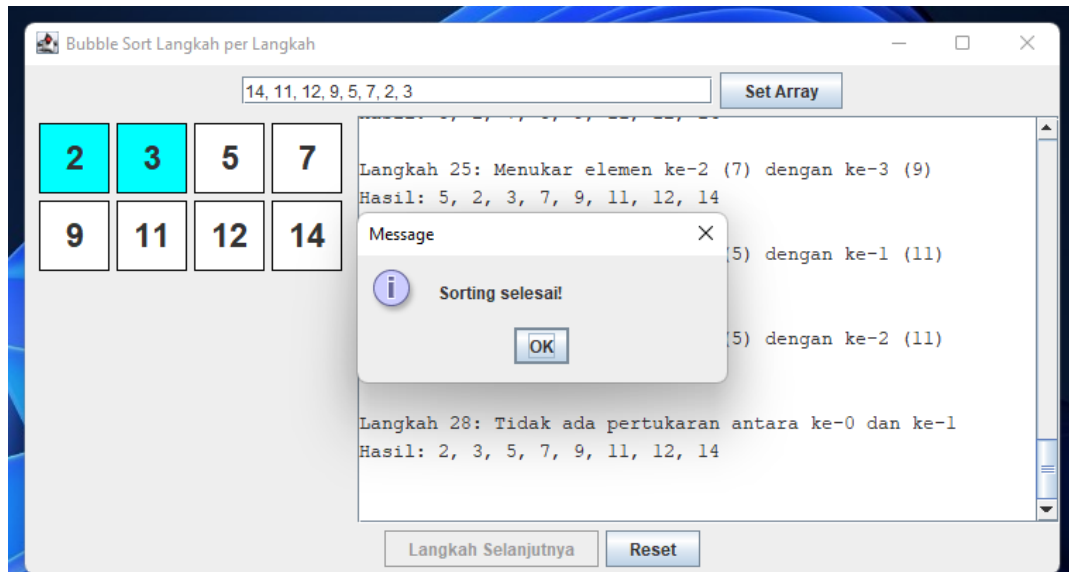
Sesudah terurut menggunakan merge sort:
5 6 7 11 12 13

```

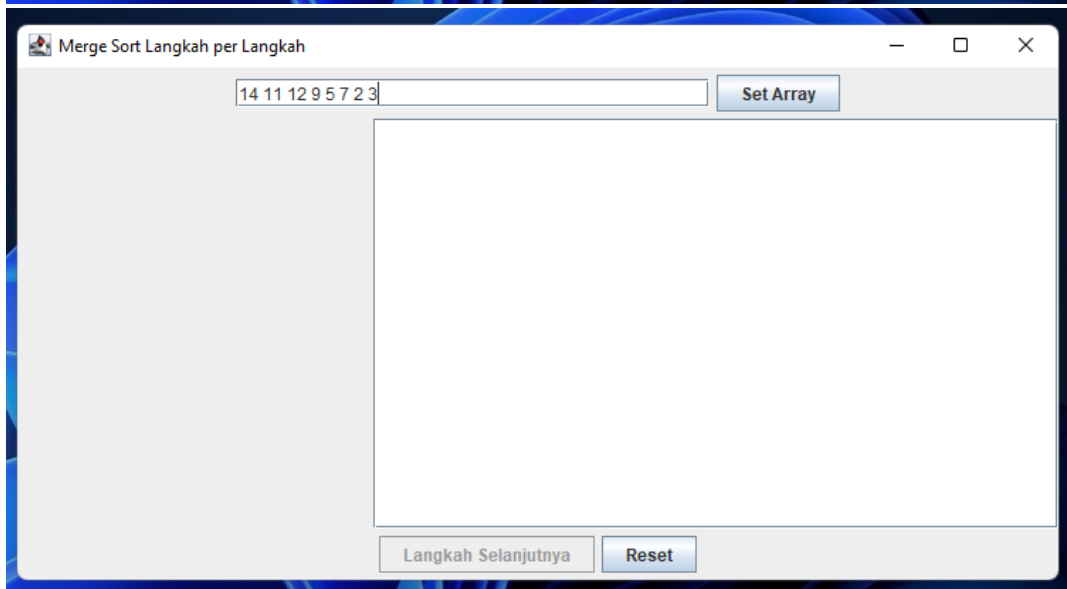
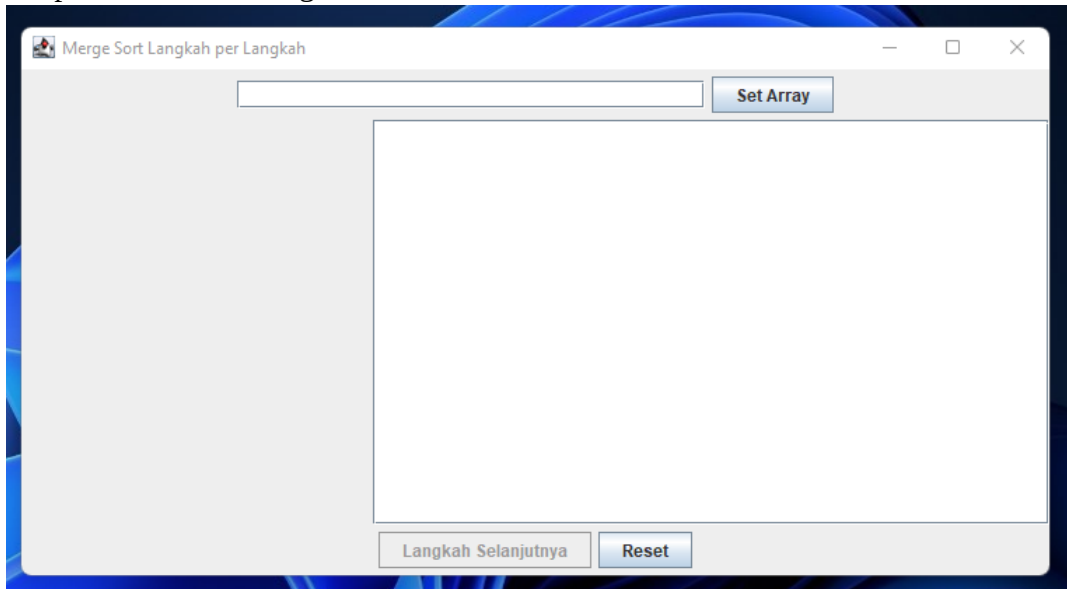
Output Kode Java BubbleSortGUI:

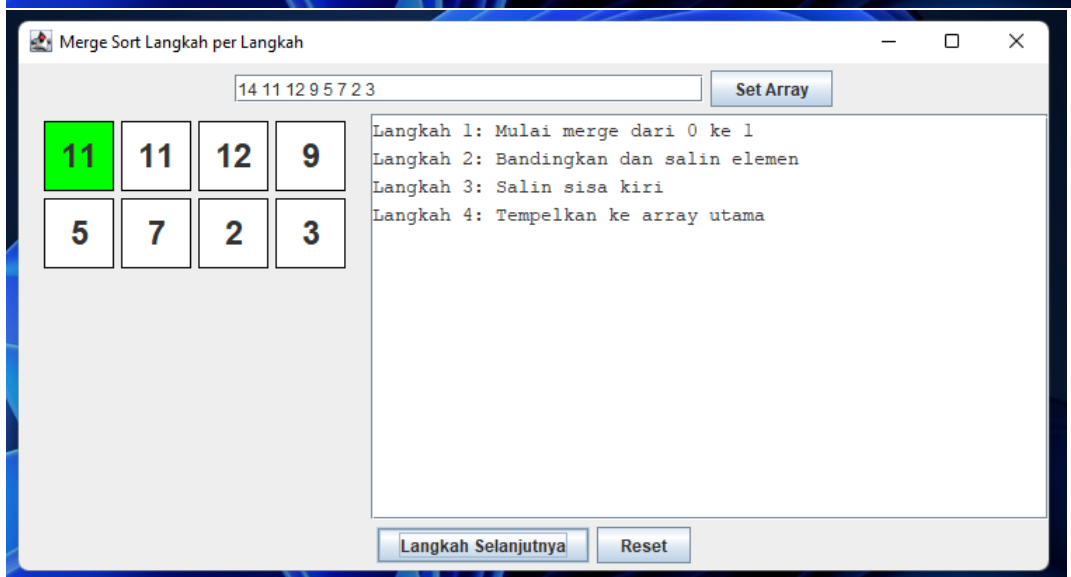
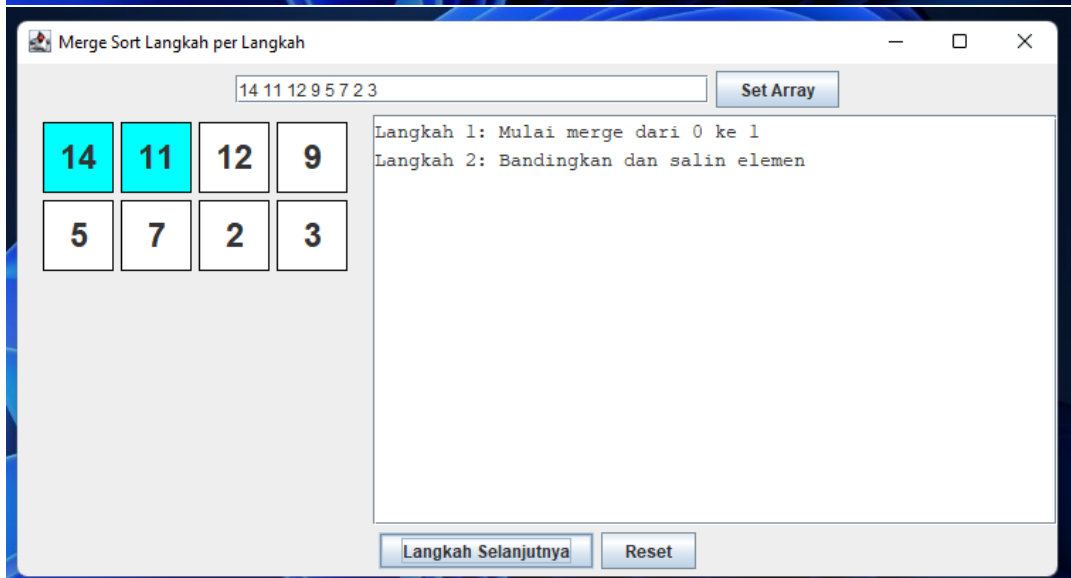
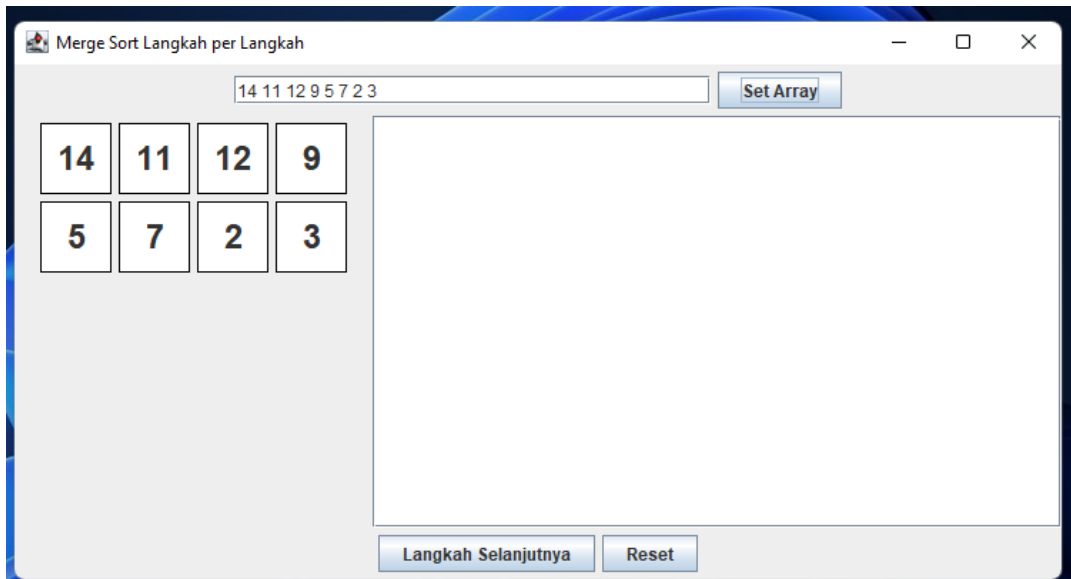


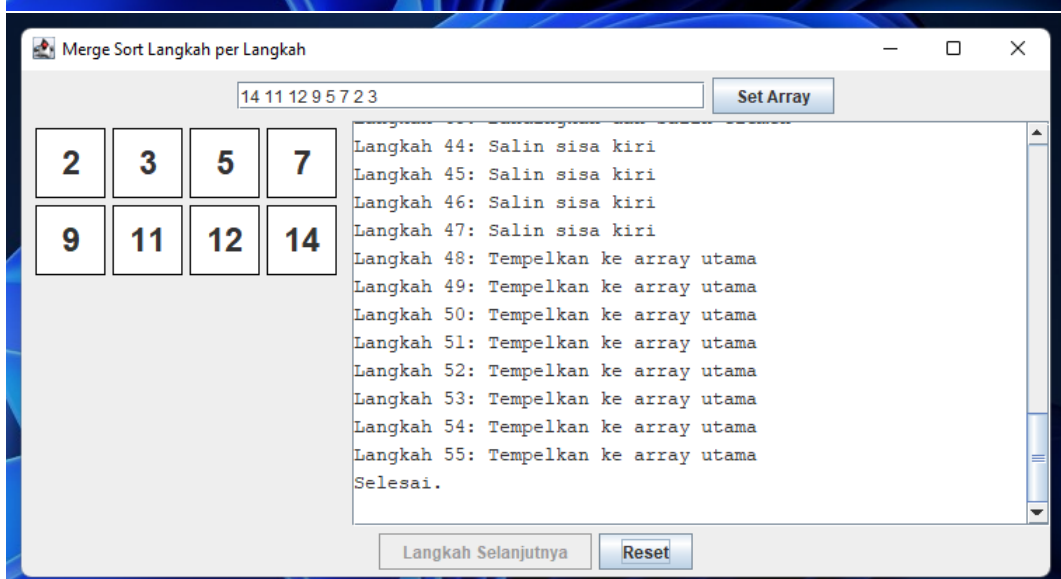
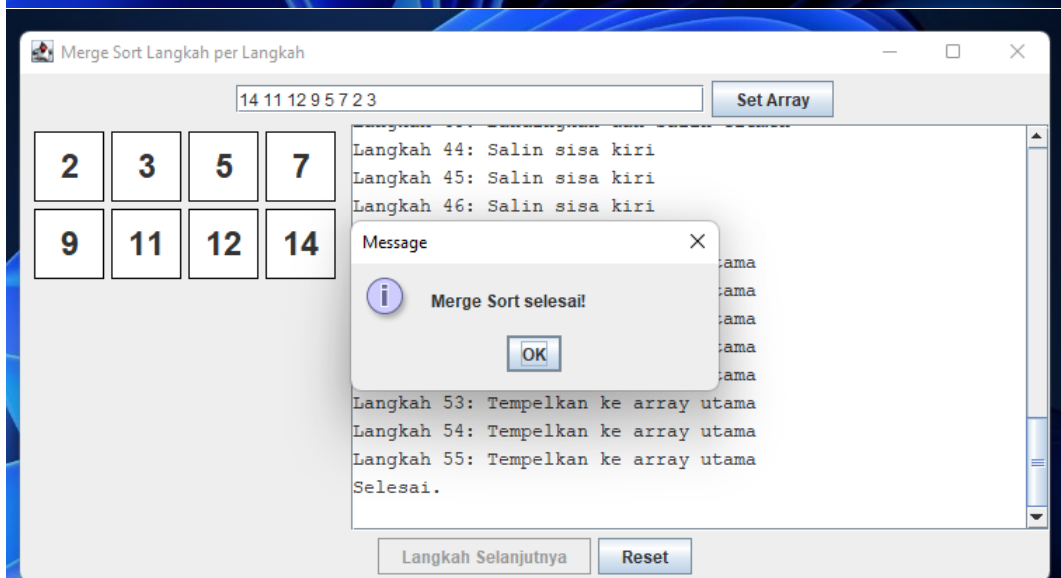
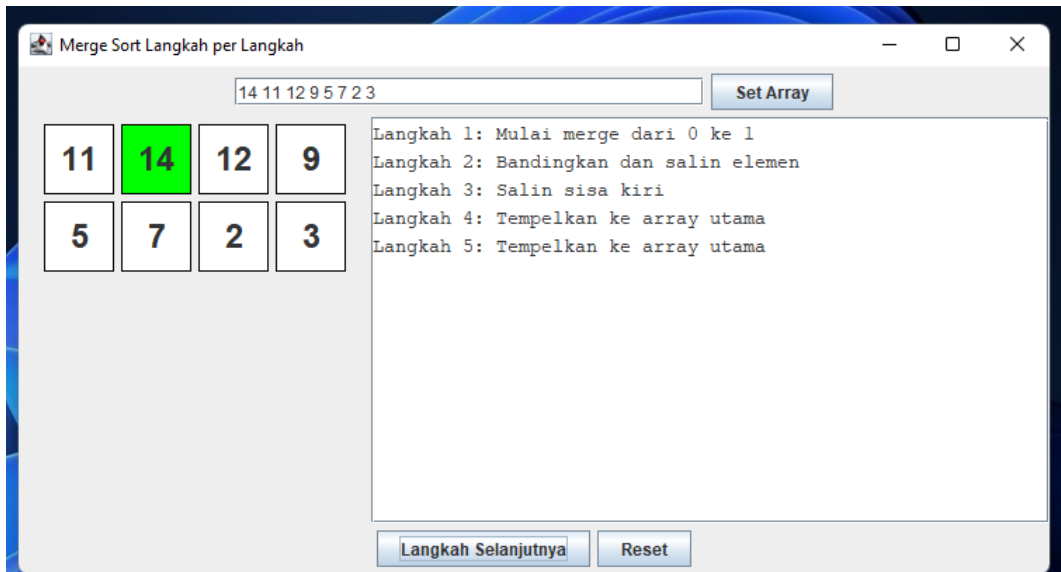




Output Kode Java MergeSortGUI:







2.4 Pembahasan

Praktikum ini menunjukkan bahwa Bahasa Java memudahkan proses membuat struktur data yang terdiri dari selector, mutator, dan lainnya. Dengan demikian, praktikum ini berhasil memberikan gambaran nyata tentang pentingnya Tipe Data Abstrak untuk struktur data dan driver sebagai eksekutor Tipe Data Abstrak untuk mendukung sistem informasi.

Bab 3 Kesimpulan

Praktikum ini berhasil menghasilkan struktur dalam data dengan menggunakan bahasa Java. Di dalam data memiliki struktur yang dapat menjalankan sebuah kasus kecil maupun besar untuk mendukung system informasi.